

Einfache Graphenalgorithmen

Eine Ausarbeitung zum Vortrag

Proseminar Kombinatorik

Sommersemester 2026

Lennart Meier

Lennart.meier05@gmail.com

Vortrag am 26.05.2026

Inhaltsverzeichnis

1	Einleitung	1
2	Grundbegriffe	1
3	Tiefensuche	2
4	Breitensuche	4
5	Bipartite Graphen	6
6	Azyklische Digraphen	7
	Literatur	9

1 Einleitung

Graphen gehören zu den wichtigsten Strukturen der diskreten Mathematik und der Informatik. Sie dienen dazu, Beziehungen zwischen Objekten darzustellen. Viele praktische Probleme lassen sich durch Graphen modellieren.

Typische Anwendungen sind:

- Straßennetze und Navigation,
- soziale Netzwerke,
- Kommunikationsnetze,
- Suchmaschinen,
- Projektplanung und Scheduling.

Ziel dieser Ausarbeitung ist die Vorstellung grundlegender Graphenalgorithmen. Dabei stehen insbesondere die Tiefensuche und die Breitensuche im Mittelpunkt. Zusätzlich werden bipartite Graphen sowie azyklische Digraphen betrachtet.

Die Ausarbeitung basiert auf [1].

2 Grundbegriffe

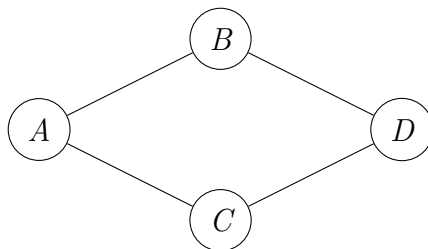
Definition 2.1. *Ein **Graph** ist ein Paar*

$$G = (V, E),$$

wobei

- *V eine endliche Menge von Knoten ist,*
- *$E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ die Menge der Kanten ist.*

Beispiel 2.2. Betrachte den folgenden Graphen:



Es gilt:

$$V = \{A, B, C, D\},$$

$$E = \{\{A, B\}, \{A, C\}, \{B, D\}, \{C, D\}\}.$$

Definition 2.3. • Ein **Weg** ist eine Folge von Knoten, bei der aufeinanderfolgende Knoten durch Kanten verbunden sind.

- Ein **Kreis** ist ein geschlossener Weg.
- Ein Graph heißt **zusammenhängend**, wenn zwischen je zwei Knoten ein Weg existiert.

Definition 2.4. Ein **gerichteter Graph** (Digraph) besteht aus:

$$G = (V, E),$$

wobei

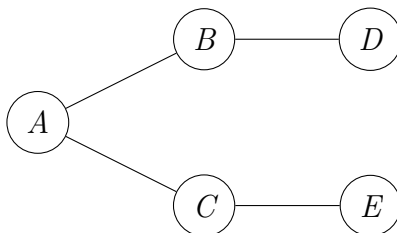
$$E \subseteq V \times V.$$

Die Kanten besitzen also eine Richtung.

3 Tiefensuche

Die Tiefensuche (*Depth First Search*, DFS) gehört zu den grundlegenden Graphenalgorithmien.

Die Idee besteht darin, möglichst tief in den Graphen vorzudringen, bevor zurückgegangen wird.



Beispiel 3.1.

Eine mögliche DFS-Reihenfolge lautet:

$A, B, D, C, E.$

Algorithmus 3.2. *Eingabe: Graph $G = (V, E)$, Startknoten s*

Ausgabe: Alle von s erreichbaren Knoten

markiere s

for alle Nachbarn v von s do

if v unmarkiert then

DFS(v)

Satz 3.3. *Die Tiefensuche besitzt Laufzeit*

$$O(|V| + |E|).$$

Beweis. Jeder Knoten wird höchstens einmal besucht. Außerdem wird jede Kante höchstens zweimal betrachtet.

Damit ist die Gesamtzahl der Operationen proportional zu

$$|V| + |E|.$$

□

Satz 3.4. *Die Tiefensuche besucht genau die Knoten der Zusammenhangskomponente des Startknotens.*

Beweis. DFS besucht nur Knoten, die über einen Weg vom Startknoten erreichbar sind.

Sei umgekehrt v erreichbar. Dann existiert ein Weg

$$s = v_0, v_1, \dots, v_k = v.$$

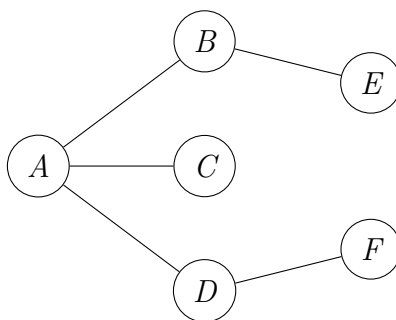
Da DFS rekursiv alle Kanten verfolgt, werden nacheinander alle Knoten dieses Weges besucht. Insbesondere wird somit auch v besucht. $\square$

4 Breitensuche

Die Breitensuche (*Breadth First Search*, BFS) durchsucht den Graphen schichtweise.

Zunächst werden alle Nachbarn des Startknotens besucht, danach die Nachbarn dieser Knoten usw.

Dabei verwendet BFS eine Queue.



Beispiel 4.1.

Die BFS-Ebenen lauten:

$$\{A\}, \{B, C, D\}, \{E, F\}.$$

Algorithmus 4.2. *Eingabe: Graph $G = (V, E)$, Startknoten s*

Ausgabe: Distanzen aller erreichbaren Knoten

```
markiere  $s$ 
 $Q \leftarrow \{s\}$ 
while  $Q \neq \emptyset$  do
     $u \leftarrow$  erstes Element von  $Q$ 
    for alle Nachbarn  $v$  von  $u$  do
        if  $v$  unmarkiert then
            markiere  $v$ 
             $Q \leftarrow Q \cup \{v\}$ 
```

Satz 4.3. *Die Breitensuche berechnet in ungewichteten Graphen kürzeste Wege vom Startknoten zu allen erreichbaren Knoten.*

Beweis. BFS arbeitet schichtweise.

Ebene k enthält genau die Knoten mit Distanz k vom Startknoten.

Alle Knoten einer Ebene werden vollständig bearbeitet, bevor Knoten der nächsten Ebene betrachtet werden.

Somit wird jeder Knoten zuerst über einen kürzesten Weg erreicht. $\square$

Satz 4.4. *Die Breitensuche besitzt Laufzeit*

$$O(|V| + |E|).$$

Beweis. Wie bei der Tiefensuche wird jeder Knoten höchstens einmal besucht und jede Kante höchstens zweimal betrachtet.

Daher ergibt sich insgesamt Laufzeit

$$O(|V| + |E|).$$

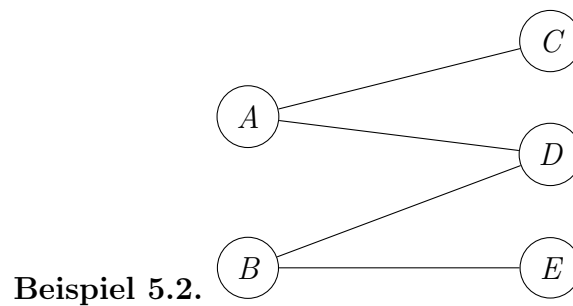
$\square$

5 Bipartite Graphen

Definition 5.1. Ein Graph heißt **bipartit**, wenn sich die Knotenmenge in zwei disjunkte Mengen

$$V_1, V_2$$

zerlegen lässt, sodass jede Kante einen Knoten aus V_1 mit einem Knoten aus V_2 verbindet.



Hier gilt:

$$V_1 = \{A, B\}, \quad V_2 = \{C, D, E\}.$$

Satz 5.3. Ein Graph ist genau dann bipartit, wenn er keinen ungeraden Kreis enthält.

Beweis. Sei zunächst G bipartit mit Zerlegung

$$V = V_1 \cup V_2.$$

Beim Durchlaufen eines Kreises wechselt man nach jeder Kante zwischen V_1 und V_2 .

Somit besitzt jeder Kreis gerade Länge.

Also existiert kein ungerader Kreis.

Sei umgekehrt G ein Graph ohne ungeraden Kreis.

Wähle einen Startknoten s .

Definiere:

$$V_1 = \{\text{Knoten mit gerader Distanz zu } s\},$$

$$V_2 = \{\text{Knoten mit ungerader Distanz zu } s\}.$$

Existierte eine Kante innerhalb einer Klasse, entstünde ein ungerader Kreis.

Folglich ist G bipartit. □

Algorithmus 5.4. (*BFS-basierter Bipartitheitstest*)

Färbe den Startknoten rot

Starte BFS

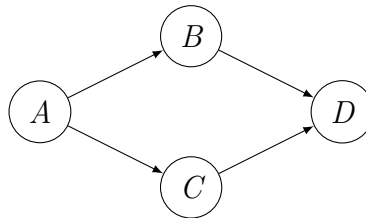
Alle Nachbarn erhalten die andere Farbe

*Treten zwei benachbarte Knoten gleicher Farbe auf,
so ist der Graph nicht bipartit.*

6 Azyklische Digraphen

Definition 6.1. Ein gerichteter Graph heißt **azyklisch**, wenn er keinen gerichteten Kreis enthält.

Solche Graphen heißen auch DAGs (*Directed Acyclic Graphs*).



Beispiel 6.2.

Definition 6.3. Eine **topologische Sortierung** eines DAGs ist eine Anordnung der Knoten

$$v_1, v_2, \dots, v_n,$$

sodass für jede gerichtete Kante

$$(v_i, v_j)$$

gilt:

$$i < j.$$

Typische Anwendungen sind:

- Projektplanung,
- Scheduling,
- Compilerbau,
- Abhängigkeitsanalysen.

Satz 6.4. *Jeder azyklische Digraph besitzt eine topologische Sortierung.*

Beweis. In jedem DAG existiert mindestens ein Knoten ohne eingehende Kante.

Entfernt man diesen Knoten, bleibt wieder ein DAG zurück.

Durch wiederholtes Entfernen solcher Knoten entsteht eine topologische Ordnung. $\square$

Literatur

- [1] Stefan Hougardy und Jens Vygen. Algorithmische Mathematik. 2., korrigierte und erweiterte Auflage. Berlin: Springer Spektrum, 2018.